```
this.elements().forEach(element => {
  option.hidden = !validValues.includes(element.value)
})
```

The intent of the code was to have `existingValidValues` return an array of strings, append `New` to that array, and then check that each element's value was in that array using the `includes` method.

However, an array `+` a string does not equal an array in JavaScript (it does in Ruby). Instead, it returns a string:

```
> ["1", "2"] + "3"
"1,23"
```

This caused my `includes` method to behave incorrectly—it was testing to see if the value was a substring of the resulting string, which is not exactly the same as seeing if the value was an element of the array.

TypeScript did not catch this issue because it assumed that I meant what I said on the first line, and since the expression as I wrote it returns a string, it assumed that I meant `validValues` to be a string. Which would have been fine under normal circumstances. It would have caught `validValues.includes` as a type error, but as it happens `includes` is a method on both string and array in JavaScript, so it didn't notice the type change.

Had I wanted to ensure that the value was of the type I expected, I could have changed that first line to have a type annotation:

```
const validValues: string[] = this.existingValidValues() + "New"
```

The annotation `: string[]` tells TypeScript that we expect the `validValues` to be set to an array of strings. We can use type annotations to describe the left-hand side of any variable declaration. Since the expression as written returns a string, the compiler would throw an error. The general use of `:` followed by a type is going to be the way in which we denote type annotations.

The fix to the code looked like this: